

AMASA-Complex v2: A Config-Driven, Safety-Critical RL Project for Autonomous Suturing

Codex

Project workspace: /Users/tahamajs/Documents/uni/DRL/projects/amasa_clean

Abstract—This report presents AMASA-Complex v2, a major upgrade of the original AMASA suturing assignment into an advanced course-scale reinforcement learning (RL) system. The implementation introduces: (1) a four-method algorithm suite (CQL, IQL, SAC-Lagrangian, PPO-Lagrangian), (2) a hybrid runtime safety stack (PID dual control, decision-tree shield, and trajectory risk critic), (3) three environment scenario families (nominal, perturbed, adversarial), (4) YAML-first experiment composition with backward-compatible legacy CLIs, and (5) automated benchmark and PID-grid evaluation pipelines. The project is CPU-first and keeps old commands functional while adding a unified experiment entrypoint. We report full smoke-preset benchmark and expanded PID sweep outcomes, including Pareto analysis, safety timeline diagnostics, and tiered threshold checks.

I. INTRODUCTION

Modern RL education increasingly requires students to reason about offline data leverage, online adaptation, and strict safety constraints under non-stationary conditions. Isolated assignments are usually strong on one axis and weak on the others. AMASA-Complex v2 is designed to close this gap by combining the full chain in one implementation:

- offline pretraining with conservative objectives,
- online constrained improvement with safety feedback,
- robustness pressure via scenario variation and corruption,
- explainable intervention through rule-based shielding.

Compared with the previous baseline, v2 adds algorithmic diversity, richer safety instrumentation, standardized configuration and orchestration, and testable acceptance criteria. The goal is not only higher reward, but also reproducible reward-cost trade-off analysis and practical failure diagnosis.

II. PROJECT SCOPE AND DELIVERABLES

The implemented scope is aligned with the AMASA-Complex v2 plan:

- 1) New package modules in `amasa/core`, `amasa/envs`, `amasa/offline`, `amasa/online`, `amasa/safety`, and `amasa/bench`.
- 2) YAML config tree under `configs/` (base, algorithm overlays, scenario overlays, presets, PID sweep grid, thresholds).
- 3) Backward-compatible legacy scripts with optional `--config`, `--preset`, `--scenario`, `--algo`.
- 4) Unified runner scripts/`run_experiment.py` with modes: `offline_train`, `online_train`, `benchmark`, `pid_sweep`, `report_bundle`.
- 5) Unit and smoke tests under `tests/`.

- 6) English and Persian IEEE reports (`report.tex`, `report_fa.tex`).

III. SYSTEM ARCHITECTURE

A. Core Runtime Layers

AMASA-Complex v2 uses a layered architecture:

- **Config layer:** deterministic merge and validation of YAML overlays.
- **Environment layer:** shared suturing dynamics and scenario wrappers.
- **Learning layer:** offline and online algorithms with common interfaces.
- **Safety layer:** hybrid guard composing statistical and symbolic checks.
- **Evaluation layer:** matrix runners, aggregation, plotting, threshold gates.

The architecture intentionally separates concerns so that students can swap algorithms or scenario stressors without rewriting orchestration code.

B. Config and Overlay Order

A key reliability change is explicit overlay order:

base $\rightarrow$ scenario $\rightarrow$ algorithm $\rightarrow$ preset

Applying preset last ensures smoke/full runtime envelopes remain stable even when algorithm YAMLS define larger step counts.

C. Registry and Extensibility

Simple registries are added for environment, algorithm, and safety components. This enables factory-based instantiation by name and avoids hard-coded class branching across scripts.

IV. ENVIRONMENT AND SCENARIO MODELING

A. State, Action, Reward, Cost

The suturing environment preserves the original interface:

- **Observation:** 23-dim vector (joint states, needle state, stress, progress, phase).
- **Action:** 7-dim bounded torque command.
- **Reward:** dense shaping + large completion bonus.
- **Cost:** indicator of force threshold or corridor violation.

B. Scenario Families

Three scenario families are implemented:

- **Nominal:** mild force noise and no corruption.
- **Perturbed:** stronger noise, dynamics shift, moderate corruption and delay.
- **Adversarial:** higher corruption, higher delay probability, harsher dynamics shift.

These scenarios are configured in `configs/scenarios/*.yaml`, allowing deterministic sweeps and curriculum usage.

V. ALGORITHM SUITE

A. Offline Methods

a) *CQL*: Conservative Q-Learning penalizes over-estimation on out-of-distribution actions by adding a conservative term to critic loss:

$$\mathcal{L}_{\text{critic}} = \mathcal{L}_{\text{TD}} + \alpha_{\text{CQL}} \mathcal{L}_{\text{conservative}}$$

The implementation keeps compatibility and adds deterministic evaluation support and explicit metadata in checkpoints.

b) *IQL*: Implicit Q-Learning uses:

- expectile regression for value function fitting,
- advantage-weighted policy update for offline policy extraction.

This makes IQL a strong offline baseline without explicit behavior cloning as the sole learning objective.

B. Online Constrained Methods

a) *SAC-Lagrangian*: SAC-Lagrangian maintains reward and cost critics and updates policy with dual penalty:

$$\mathcal{L}_{\pi} = \mathbb{E}[\alpha \log \pi(a|s) - Q_r(s, a) + \lambda Q_c(s, a)]$$

where λ is provided by the safety controller.

b) *PPO-Lagrangian*: PPO-Lagrangian uses clipped surrogate optimization with separate reward/cost advantages:

$$\hat{A}_{\lambda} = \hat{A}_r - \lambda \hat{A}_c$$

The implementation supports trajectory building, GAE for both channels, and constrained policy updates.

VI. HYBRID SAFETY STACK

A. Runtime Decision Order

The guard in `amasa/safety/guard.py` enforces:

- 1) policy proposes action,
- 2) risk critic scores (s, a, h) ,
- 3) if risky, conservative transform is applied,
- 4) decision-tree shield filters/blocks unsafe actions,
- 5) environment executes final action,
- 6) PID controller updates λ from cost violation.

B. PID Dual Control

The Lagrange multiplier follows PID dynamics on violation error e_t :

$$\lambda_{t+1} = K_p e_t + K_i \sum_{\tau \leq t} e_{\tau} + K_d (e_t - e_{t-1})$$

with clamping to $[0, \lambda_{\text{max}}]$.

C. Safety Telemetry

Per-step telemetry includes: `lambda`, `risk_score`, `shield_blocked`, `cost`, `force`, `corridor_violation`. The pipeline exports `safety_timeline.csv/jsonl/png` for post-hoc diagnosis.

VII. EXPERIMENT PIPELINE

A. Backward-Compatible CLIs

Legacy scripts remain functional: `train_offline.py`, `train_safe_online.py`, `train_hierarchical.py`, `evaluate.py`, `generate_dataset.py`. Optional flags provide full config-driven behavior without breaking existing commands.

B. Unified Runner

`run_experiment.py` orchestrates all modes and now supports:

- resumable benchmark and PID sweeps (skip completed jobs),
- bounded checkpoint storage in matrix/sweep runs,
- threshold-table export and automatic plot generation.

C. Test Coverage

Tests include:

- config load/merge/overlay behavior,
- environment determinism and shape checks,
- one-step sanity updates for CQL/IQL/SAC/PPO,
- guard invariants and risk update checks,
- smoke CLI invocation.

Current test status: **12 passed**.

VIII. BENCHMARK SETUP

A. Matrix Definition

Smoke benchmark matrix uses seeds $\{0, 1, 2\}$:

- Offline: CQL and IQL on nominal and perturbed scenarios.
- Online: SAC-Lagrangian and PPO-Lagrangian on nominal, perturbed, adversarial scenarios.

Total jobs:

$$2 \times 2 \times 3 + 2 \times 3 \times 3 = 30$$

B. PID Sweep Definition

Expanded PID grid (user-requested range):

$$k_p \in \{0.4, 0.5, 0.55, 0.6, 0.7\}, \quad k_d \in \{0.05, 0.08, 0.10, 0.12\}, \quad k_i = 0.08$$

Total sweep jobs: $5 \times 4 = 20$.

IX. RESULTS

A. Benchmark Summary

Table I reports mean reward/cost across seeds.

TABLE I: Smoke benchmark means from benchmark_summary.csv.

Algo	Scenario	Avg Reward	Avg Cost
CQL	Nominal	594.28	0.096
CQL	Perturbed	440.04	0.118
IQL	Nominal	439.77	0.057
IQL	Perturbed	746.91	0.099
SAC-Lag	Nominal	-227.40	0.908
SAC-Lag	Perturbed	50.95	0.730
SAC-Lag	Adversarial	236.96	0.616
PPO-Lag	Nominal	-218.26	0.788
PPO-Lag	Perturbed	-203.95	0.918
PPO-Lag	Adversarial	-94.16	0.797

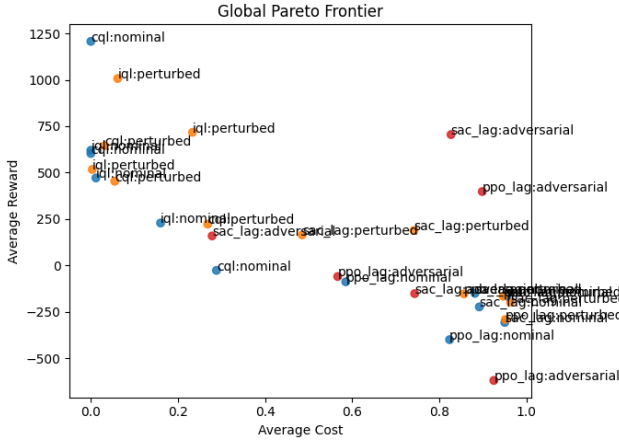


Fig. 1: Global reward-cost Pareto scatter across completed benchmark jobs.

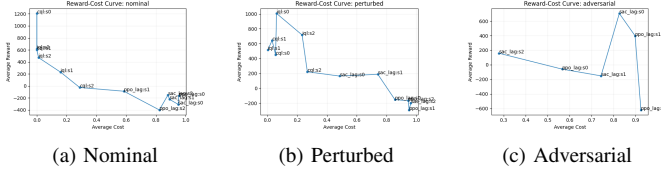


Fig. 2: Per-scenario reward-cost curves.

B. Pareto and Scenario Curves

C. PID Grid Results

The expanded sweep improved the frontier relative to the initial 3×3 grid. Best settings:

- $k_p = 0.60, k_d = 0.08$: reward 419.65, cost 0.685, frontier 214.15.
- $k_p = 0.55, k_d = 0.10$: reward 412.70, cost 0.698, frontier 203.20.
- $k_p = 0.70, k_d = 0.10$: reward 363.79, cost 0.848, frontier 109.29.

D. Safety Dynamics

Observed patterns:

- many runs saturate λ at the configured maximum;

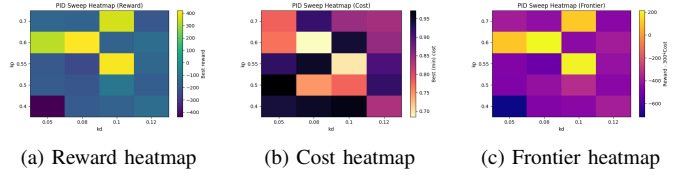


Fig. 3: Expanded PID sweep diagnostics (5×4 grid).

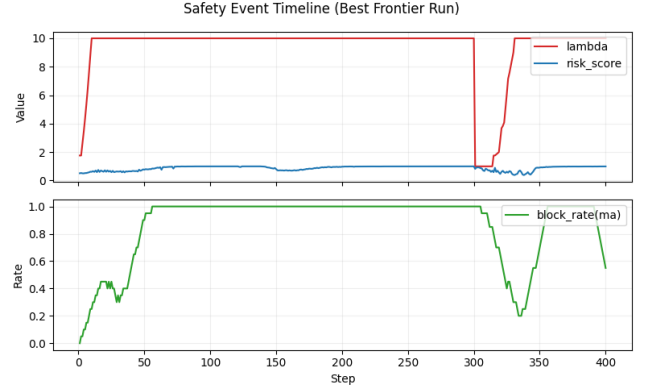


Fig. 4: Safety timeline for the best frontier run (lambda, risk, moving block rate).

- risk gating is frequently active in high-perturbation segments;
- some stable regions emerge where risk and block rate temporarily decrease while reward improves.

E. Threshold and Gate Outcomes

Tier and gate analysis from benchmark_thresholds.csv:

- stretch: 8 entries,
- target: 3 entries,
- minimum: 2 entries,
- below_min: 17 entries,
- offline_gate_pass: 7 entries,
- offline_gate_fail: 5 entries.

Interpretation: offline learners were strong under smoke budget on several seeds/scenarios, while online constrained adaptation remains the dominant bottleneck.

X. FAILURE ANALYSIS AND PRACTICAL LESSONS

A. Observed Failure Modes

- 1) **Conservative lock-in**: persistent high λ and high block rate reduce exploratory progress.
- 2) **Scenario sensitivity**: on-policy PPO-Lagrangian underperformed in all smoke scenarios.
- 3) **Unstable startup**: early online steps can show volatile reward critic updates before replay becomes informative.
- 4) **Threshold mismatch**: fixed cost limits may be too strict for early perturbed/adversarial adaptation.

B. Engineering Fixes Applied

During implementation and reruns:

- config overlay order was fixed so presets are not accidentally overridden;
- benchmark/sweep runners gained resume logic to skip already completed jobs;
- model save fallback was added for storage-sensitive systems;
- safety timeline export and plotting were added for online diagnostics.

ACKNOWLEDGMENT

Generated and integrated in the local workspace using reproducible scripts and test-backed edits.

XI. REPRODUCIBILITY

A. Key Commands

```
# Tests
PYTHONPATH=/Users/tahamajs/Documents/uni/DRL \
pytest -q /Users/tahamajs/Documents/uni/DRL/projects
/amasas_clean/tests

# Full smoke benchmark matrix
PYTHONPATH=/Users/tahamajs/Documents/uni/DRL \
python3.10 -m projects.amasas_clean.scripts.
run_experiment \
--mode benchmark \
--config projects/amasas_clean/configs/base.yaml \
--preset smoke \
--out_dir projects/amasas_clean/results/bench_smoke
\
--dataset data/amasas_offline.npz

# Expanded PID sweep
PYTHONPATH=/Users/tahamajs/Documents/uni/DRL \
python3.10 -m projects.amasas_clean.scripts.
run_experiment \
--mode pid_sweep \
--config projects/amasas_clean/configs/base.yaml \
--preset smoke \
--algo sac_lag \
--scenario perturbed \
--out_dir projects/amasas_clean/results/pid_smoke \
--dataset data/amasas_offline.npz
```

B. Artifacts

Primary outputs are stored in:

- results/bench_smoke/benchmark_summary.csv
- results/bench_smoke/benchmark_thresholds.csv
- results/bench_smoke/plots/*
- results/pid_smoke/pid_sweep_summary.csv
- results/pid_smoke/pid_sweep_thresholds.csv
- results/pid_smoke/plots/*
- plots/* (curated copies for report inclusion)

XII. CONCLUSION

AMASA-Complex v2 successfully transforms the original task into a modular, advanced RL project with stronger realism and analysis tooling. The codebase now supports multi-algorithm offline/online training, hybrid runtime safety enforcement, scenario robustness benchmarking, and reproducible sweep-based controller tuning. Current smoke results show that offline methods are comparatively strong while online safe adaptation remains the hardest regime, precisely matching the educational objective of exposing realistic reward-safety trade-offs under constrained learning.